

Prompt 设计指南

Akiro

本指南面向AI Agent、代码生成、内容处理等实际工程场景，提供可量化的设计方法和迭代流程。Prompt是向模型传递任务意图的主要载体，其设计质量直接决定输出结果的可用性。

1. Prompt的组成结构

一个完整的Prompt由以下五个功能模块构成，开发者可根据任务类型自由裁剪组合。

模块	功能定义	写法示例
角色设定	限定模型的视角和专业领域	“你是后端系统架构师，熟悉Java并发编程”
任务描述	明确要完成的具体工作	“设计一个支持千万级QPS的限流器伪代码”
上下文信息	提供背景数据、业务规则、历史记录	“现有系统基于Spring Cloud，网关层未做防护”
输出规范	指定格式、长度、结构、语言风格	“输出Markdown表格，包含方案名、优缺点、适用条件”
交互指引	规定推理路径或要求分步输出	“先分析流量峰值特征，再给出方案，最后说明降级策略”

设计原则：每个模块只承担单一职责。如果任务简单，可以合并或删除部分模块。避免在一个句子中混杂多个模块的内容。

2. 设计准则（工程向）

2.1 正向指令优先于反向限制

模型对“应该做什么”的响应准确率高于“不要做什么”。将限制条件转化为正向行为描述。

错误写法：“不要使用口语，不要出现主观评价，不要省略技术细节。”

正确写法：“使用技术文档的客观表述，列出关键参数和边界条件。”

2.2 显式化思维路径

对复杂任务，要求模型输出推理步骤。这能有效减少中间逻辑跳跃导致的结论偏差。
强制写法：“按以下顺序作答：第一步，列出所有输入约束；第二步，提出至少两种方案；第三步，给出推荐方案及其理由。”

2.3 示例选择的量化规则

少样本示例的数量控制在3至6个。分类任务中，每个类别的示例数量需均衡。示例排序应遵循难度递增或典型性递减的原则，防止模型对首尾示例产生位置过拟合。

2.4 长对话中的锚点重置

Transformer架构对上下文开头和末尾的注意力权重更高。对话长度超过4000个token时，需在每次新提问的末尾重复一次核心目标。使用[核心目标]或[硬性要求]作为前缀标记，强化关键信息的注意力分数。

3. 迭代优化流程

采用以下标准化流程进行Prompt调优，每次修改需记录版本日志。

步骤	操作内容	记录项
基线构建	根据任务拆解写出初版Prompt	初始版本号、预期输出样例
冒烟测试	用3条典型输入执行，检查输出是否完整	通过/失败项、错误类型标签
归因分析	将错误分为语义误解、格式偏离、信息遗漏三类	错误分布统计
定向修正	每次只调整一个模块，优先修改指令描述	修改前后对比
回归测试	使用相同测试集重新执行，比对改进幅度	准确率、格式合规率变化
固化发布	输出稳定后编写使用说明文档	适用边界、已知缺陷说明

4. 网页问答场景下的Prompt设计要点

网页问答（Web Q&A）指模型基于给定的网页内容或搜索结果回答用户问题。该场景的核心风险是模型依赖自身参数化记忆而产生幻觉。Prompt需强制模型绑定外部输入。

4.1 上下文注入格式

将网页内容与问题在结构上强制隔离，明确限定引用范围。

示例结构：

[可用资料]

（此处粘贴抓取到的网页纯文本内容，保留段落标记）

[资料结束]

[用户问题]

（在此处写入用户的具体提问）

[回答规则]

- 只使用[可用资料]中的信息作答。
- 如果资料中找不到答案，直接回复“根据现有资料无法确认”，不补充外部知识。
- 引用资料中的具体段落时，在句末标注原文段落编号。

4.2 不确定性处理

必须预设资料不完整的情况。在回答规则中加入置信度自检指令：

“在回答末尾，标注答案置信度（高/中/低）。置信度为中或低时，需明确指出资料中缺失的关键信息项。”

4.3 多轮追问的上下文承接

在连续问答中，每次输出需附带当前轮次的“回答依据摘要”，便于下一轮模型快速定位。

输出格式示例：

“回答：……。依据摘要：引用段落3、5。”

5. 大型项目开发场景下的Prompt设计（适配OpenCode等Agent工具）

当使用OpenCode、Cursor、Devin等AI编程Agent开发大型工程项目（代码量超过5000行、涉及多模块、多依赖）时，Prompt设计不能以“单次任务”为单位，而需以“项目迭代”为单位重构。OpenCode这类工具执行时会将Prompt拆解为子任务并调用多个上下文窗口，因此Prompt需要提供稳定的边界约束。

5.1 项目级系统指令（Project-level System Prompt）

在OpenCode的工作目录中，需放置一个持久化的系统指令文件

（如 `.opencode/instructions.md`）。该文件规定了Agent在整个项目生命周期内的全局

行为，不受单次对话影响。

必填内容：

- 编程语言及版本（如Python 3.11、Java 17）
- 包管理器及镜像源（如pnpm、npm registry）
- 测试框架及覆盖率要求（如Jest、Pytest，行覆盖率不低于80%）
- 代码风格规范（如ESLint规则、Google Java Format）
- 文档语言（中文/英文）及注释密度（每函数至少一行注释）

5.2 模块级任务拆解指令（Module-level Task Decomposition）

不要一次性提交整个项目的需求，这会导致Agent上下文溢出或关键依赖遗漏。正确的做法是分模块提交，但Prompt中需提供“模块间依赖声明”和“接口契约”。

标准写法：

[当前任务]

实现用户认证模块的登录接口（`/api/v1/login`）。

[模块边界]

- 仅处理登录逻辑，不涉及注册、密码重置。
- 依赖项：数据库连接池（已存在，路径：`src/db/pool.ts`）。
- 对外暴露接口：返回JWT token及用户基本信息。

[上游契约]

- 调用方（前端）期望响应结构：`{ code: number, data: { token: string, expires_in`
- 错误码范围：`1001-1005`。

[下游依赖]

- 用户表结构定义见 `src/models/user.sql`。
- 缓存服务（Redis）需更新登录失败次数，key格式：`login_fail:{ip}:{username}`。

[验收标准]

- 密码错误3次后锁定账号15分钟。
- 单元测试覆盖正常登录、密码错误、账号锁定三种情况。
- 接口响应时间中位数小于200ms。

[输出约束]

- 只修改 `src/controllers/auth.ts` 和 `src/services/auth.service.ts`。
- 不修改配置文件，不新增外部依赖。
- 生成代码后，附带一条pytest执行命令。

5.3 存量代码修改指令（Change Request Prompt）

在大型项目中，Agent经常需要修改现有逻辑。此时Prompt必须包含“修改范围锁定”和

“影响分析”。

写法示例：

[修改目标]

将 `src/utils/validator.ts` 中的邮箱正则表达式由 `patternA` 替换为 `patternB`（支持新语

[影响范围分析]

- 调用此验证器的模块有：注册接口、个人信息修改接口、找回密码接口。
- 请检查这三个调用点在新的正则下的兼容性，并在修改后更新对应的单元测试用例。

[回归检查]

- 运行 `npm run test:validator` 确保原有测试用例（除刻意修正的）不出现新失败。
- 输出修改前后的匹配差异列表（至少列出5个边界用例，如 `".io"`、`".ai"` 等新后缀）。

5.4 错误诊断与修复指令（Debug Prompt）

当Agent需要定位bug时，Prompt应直接附上完整的错误堆栈和最近一次变更的commit hash，并要求Agent给出根因分析和修复方案对比。

标准格式：

[错误现象]

生产环境报错： `TypeError: Cannot read property 'id' of undefined`，堆栈指向 `src/r`

[上下文]

该文件在 `commit 7f3a2b1` 中重构了 `token` 解析逻辑。

[诊断要求]

1. 列出导致此错误的可能代码路径（至少两种）。
2. 分别说明每种路径触发的条件。
3. 推荐修复方案并附上`patch`代码。
4. 说明如何增加防御性编程避免类似问题。

5.5 大型项目Prompt设计的避坑清单

- 不要使用“帮我优化代码”这类模糊指令。必须指明优化维度（性能/可读性/内存占用）。
- 不要将前端UI修改与数据库迁移指令放在同一个Prompt中。Agent工具在单次请求中倾向于一次性修改过多文件，增加冲突风险。
- 如果项目包含自定义脚本或Makefile，必须在Prompt中显式告知Agent“编译/构建命令为xxx，测试启动命令为xxx”。
- 对于OpenCode这类支持工具调用（Function Call）的Agent，需在Prompt中明确允许或禁止其执行终端命令、文件写入、网络请求等权限。

6. 效果评估指标

在实践中，对Prompt输出质量的评估建议采用以下四项硬指标，而非主观评分。

指标	计算方法	合格线
格式合规率	输出结构符合指定模板的占比	$\geq 95\%$
关键要素覆盖率	必需信息点是否全部出现	100%
事实幻觉率	未经证实的断言数量占总断言数的比例	$\leq 2\%$
指令遵循偏差	未执行的明确指令数量	0

7. 附录：常用前缀标记

在长Prompt或连续对话中，使用标准化前缀标记可以提升模型对关键信息的解析效率。

- [硬性约束]：不可违反的边界条件。
- [重要]：需分配额外注意力的内容。
- [可选]：非必需但期望的优化项。
- [背景]：供参考但不强制执行的上下文。
- [反例]：明确告知模型不应采用的行为或输出。